并行与高性能计算

第 1 章精读讲义

Why parallel computing?

核心段落翻译 · 原图导读 · C / OpenMP 语法

Robert Robey / Yuliana Zamora
Manning • 2021

覆盖原书正文第 3-34 页
对应原文件 PDF 第 39-70 页
保留 1.1-1.7 及其编号子小节，精选 17 幅核心原图。

阅读方法

每节先读核心段落的中英对照，再结合图注理解概念。公式保留原书符号，并说明适用条件。最后通过两个小型 C/OpenMP 示例把概念落实到循环、下标和读写依赖。

范围说明

这是一份按章交付的精读讲义，不是逐段全文译本。教学补充与原文译文明确区分。本次完成第 1 章，第 2-17 章尚未翻译。原书的硬件数据按 2021 年出版语境阅读。

制作依据

采用用户提供的 PDF，并应用 nature-reader 的来源锚点、双语对齐、图像提取与术语核对流程。附带 Markdown、来源映射、说明文件和可独立阅读的代码。

阅读导航与术语

章前导读 | 为什么要并行计算？ • 3-5

1.1 为什么要学习并行计算？ • 6-8

1.1.1 并行计算能带来什么收益？ • 8-11

1.1.2 并行计算的注意事项 • 11

1.2 并行计算的基本定律 | 1.2.1 Amdahl 定律 • 11-12

1.2.2 Gustafson-Barsis 定律与弱扩展 • 12-15

1.3 并行计算如何工作？ | 1.3.1 示例应用 • 15-18

1.3.1 示例应用（续）：逐层加入并行 • 19-21

1.3.2 异构并行系统的硬件模型 • 22-25

1.3.3 异构并行系统的软件模型 • 25-29

1.4 并行方式的分类 • 29-30

1.5 并行策略 • 30

1.6 并行加速比与对比加速比：两种不同度量 • 31-32

1.7 本书将教会你什么？ • 32-34

教学补充 A | 把“并行性”写成 C / OpenMP • 对应 1.3；非原书代码

教学补充 B | 五点模板：为什么要两份数组？ • 对应图 1.10；非原书代码

1.7.2 练习 | 题意与解题思路 • 33

先分清这些概念

parallelism → 并行性。可被同时执行的工作；parallel computing 译为并行计算。

concurrency → 并发。与实际同时执行的 parallel execution 区分。

process → 进程。软件概念，不译为处理器。

processor / CPU → 处理器 / 中央处理器。与 process 区分，核心为 core。

thread → 线程。与物理核心及逻辑处理器区分。

vectorization / SIMD → 向量化 / 单指令流多数据流。SIMD 全称 single instruction, multiple data。

SIMT → 单指令、多线程。single instruction, multi-thread。

node → 计算节点。图 1.13 的 node 指数据位置，按上下文说明。

kernel → 计算内核。与操作系统内核 kernel 区分。

stencil → 模板。本章 five-point stencil 为五点模板。

章前导读 | 为什么要并行计算？

核心段落 | 原书 p. 5 · S001

Parallel computing is the practice of identifying and exposing parallelism in algorithms, expressing this in our software, and understanding the costs, benefits, and limitations of the chosen implementation.

并行计算是一种实践：识别并显露算法中的并行性，通过软件将其表达出来，并理解所选实现方式的成本、收益和局限。

先建立一个判断顺序

先问“哪些工作能够独立完成”，再问“用线程、进程还是 GPU”。并发描述多个任务能够交错推进；并行强调多个操作在同一时段实际执行。单核可以通过调度实现并发，但并不因此获得多个物理核心的同时计算能力。算法允许并行、软件表达并行、硬件实际执行并行，是三个不同层次。

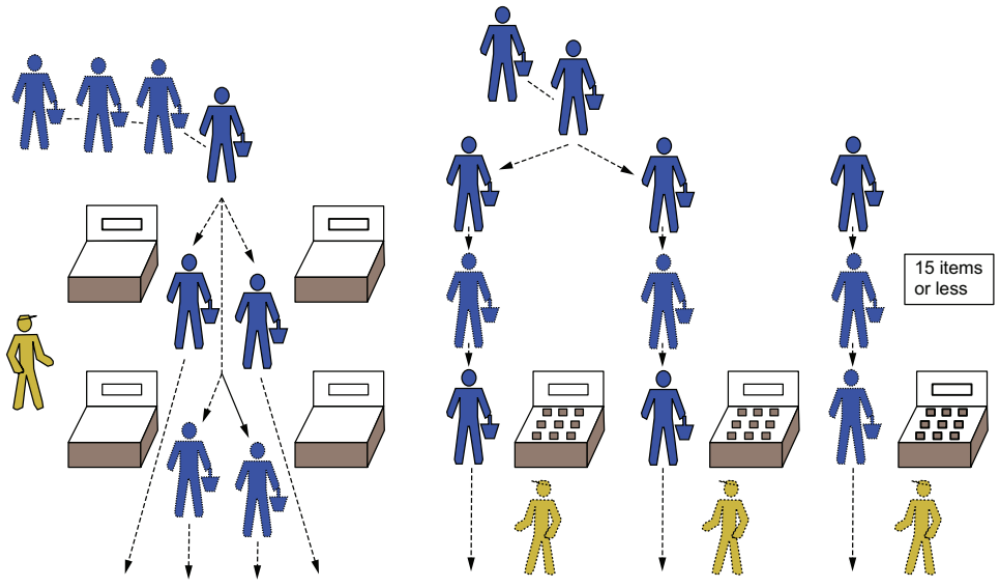


图 1.1 | 原书 p. 5

Everyday parallelism in supermarket checkout queues. The checkout cashiers (with caps) process their queue of customers (with baskets). On the left, one cashier processes four self-checkout lanes simultaneously. On the right, one cashier is required for each checkout lane. Each option impacts the supermarket’s costs and checkout rates.

图注：超市收银队列中的日常并行。戴帽子的收银员处理提篮顾客的队列。左侧一名收银员同时照管四条自助结账通道，右侧每条通道都需要一名收银员。不同方案影响超市成本和结账速率。

读图：不同顾客的结账可并行，但同一顾客的扫描、付款仍有先后关系。增加通道提高总体吞吐量，不保证每位顾客的单次结账都更快。

把比喻映射到程序

顾客相当于任务，收银通道相当于执行资源，排队规则相当于调度。某条队列特别长而其他通道闲置，就是负载不均衡。把任务拆得更细有利于分配，但调度本身也会消耗时间。

1.1 为什么要学习并行计算？

核心段落 | 原书 p.6 · S002

The future is parallel. The increase in serial performance has plateaued as processor designs have hit the limits of miniaturization, clock frequency, power, and even heat.

未来走向并行。处理器设计受到微型化、时钟频率、功耗乃至散热的限制，串行性能的增长已经进入平台期。

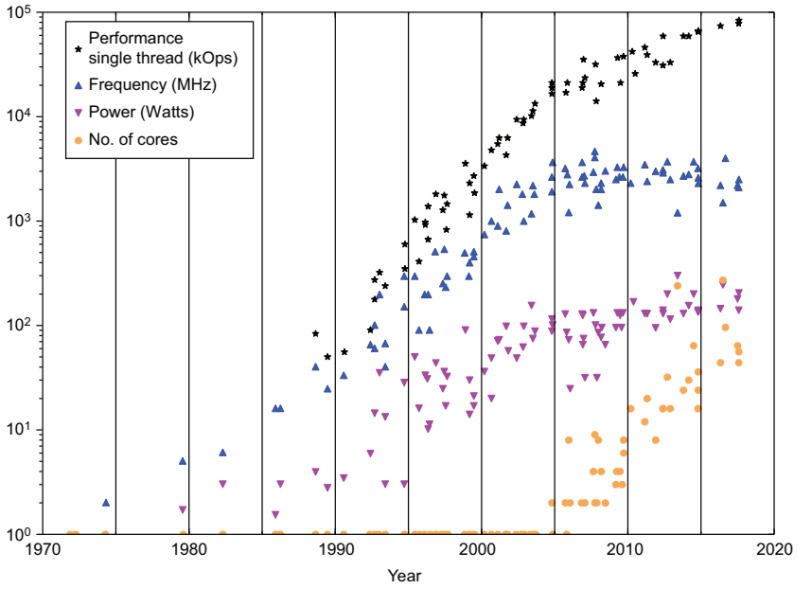


图 1.2 | 原书 p.6

Single thread performance, CPU clock frequency (MHz), CPU power consumption (watts), and the number of CPU cores from 1970 to 2018. The parallel computing era begins about 2005, when the core count in CPU chips begins to rise, while the clock frequency and power consumption plateaus, yet performance steadily increases (Horowitz et al. and Rupp, https://github.com/karlrupp/microprocessor-trend-data).

图注：1970–2018 年的单线程性能、CPU 时钟频率（MHz）、CPU 功耗（W）与 CPU 核心数。约从 2005 年开始，芯片核心数上升，频率和功耗趋于平台，而性能仍在提高。图源：Horowitz 等及 Rupp。

读图：纵轴是对数尺度，几类指标的单位也不同。读趋势而不要直接比较曲线高度。图中数据截止 2018 年，是本书的历史背景，不是 2026 年硬件选购依据。

核心、超线程和向量通道

一个物理核心内部可以有向量执行单元。256 bit 向量可容纳 4 个 64 bit double 或 8 个 32 bit float。超线程则让一个物理核心提供多个逻辑执行上下文；这些上下文共享执行资源。[R1]

原书简化提示：0.8% 不能当作实测利用率

原书第 7 页用 16 核 × 2 超线程 × 4 个 double 通道 = 128 路，进而写出串行路径约占 0.8%。这个算式可用于提示多层并行机会，但不能推出程序必然获得 128 倍加速。超线程不是新增一套完整的运算单元，内存带宽、指令吞吐、数据依赖也会限制性能。单线程程序还可能已被编译器自动向量化。

1.1.1 并行计算能带来什么收益？

核心段落 | 原书 p.8 · S003

Parallel computing can reduce your time to solution, increase the energy efficiency in your application, and enable you to tackle larger problems on currently existing hardware.

并行计算可以缩短求解时间、提高应用的能源效率，并让你利用现有硬件处理更大的问题。

更短运行时间 | Faster run time with more compute cores

同样的输入、算法要求和误差标准，比较完成任务所需的时间。例如把 10 小时缩短为 1 小时，是 10 倍加速。只有最终完成整个任务更快，才算实现了应用层面的收益。

更大问题规模 | Larger problem sizes with more compute nodes

更多节点不仅提供计算核心，也提供更多内存、存储和通信能力。原来一台机器放不下的网格，可以分布到多台机器；但每台机器都复制全量数据会削弱这种收益。

更高能源效率 | Energy efficiency by doing more with less

比较能量而不仅是功率。设备功率高但工作时间短，总能耗仍可能更低。原书用处理器数量、热设计功耗和运行时间粗估能耗；实际整机能耗还包括内存、主机、网络和空闲损耗。TDP 是设计相关指标，不能直接视为某个程序的实测平均功率。

E001 | $P = N \times R \times T$ [Wh]

原书用 P 表示能量，单位是 Wh；这里保留原符号。通常物理教材用 P 表示功率，因此阅读时要检查单位。

复算原书例题

按第 10 页给定的数值：$20 \times 120\ \text{W} \times 24\ \text{h} = 57.60\ \text{kWh}$；$4 \times 300\ \text{W} \times 24\ \text{h} = 28.80\ \text{kWh}$。两组任务的运行时间相同，第二组的设备能耗估计减半。这只是该假设下的结果，不是所有 CPU 与 GPU 的固定关系。

降低货币成本 | Parallel computing can reduce costs

如果按设备数量和使用时间计费，单价更高但运行更快的设备仍可能更便宜。评估还应计入代码迁移、验证和维护成本。

1.1.2 并行计算的注意事项

核心段落 | 原书 p.11 · S004

It is always more important that the application runs and generates the desired result before making it fast and scaling it up to larger problems.

在让应用更快、扩展到更大问题之前，更重要的始终是让它能够运行，并生成预期结果。

什么时候可能不值得并行化

很小的循环可能几微秒就结束，启动线程和同步反而更慢；串行依赖强的算法难以均匀拆分；维护成本很高而只运行一次的任务，也未必值得迁移。判断依据是预期收益和实际测量。

最小工作流程

保留可信的串行结果，测出耗时集中在哪些函数，选择最昂贵且有并行机会的部分，改动后先验结果再测性能。数值算法还要约定误差容限；浮点加法改变顺序后，末位可能发生变化。

1.2 并行计算的基本定律 | 1.2.1 Amdahl 定律

核心段落 | 原书 p.12 · S005

Amdahl's Law highlights that no matter how fast we make the parallel part of the code, we will always be limited by the serial portion.

Amdahl 定律强调：无论把代码的并行部分加速到什么程度，整体性能始终受到串行部分的限制。

E002 | SpeedUp(N) = 1 / (S + P/N)

固定总问题规模；S 是基准串行运行时间中无法并行的比例，P 是可并行比例，S + P = 1，N 是处理器数量。比例按耗时衡量，不按代码行数衡量。

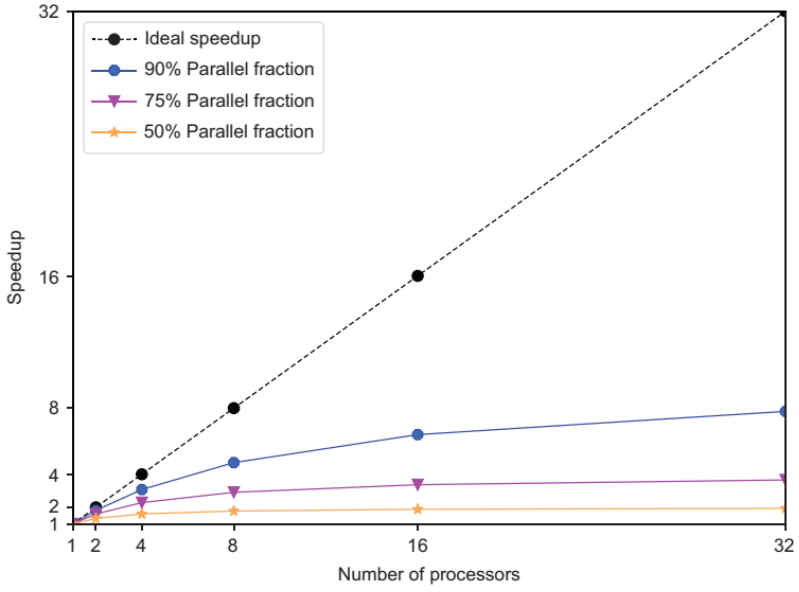


图 1.3 | 原书 p.12

Speedup for a fixed-size problem according to Amdahl's Law is shown as a function of the number of processors. Lines show ideal speedup when 100% of an algorithm is parallelized, and for 90%, 75%, and 50%. Amdahl's Law states that speedup is limited by the fractions of code that remain serial.

图注：固定问题规模时，Amdahl 定律给出的加速比随处理器数量变化。曲线对应 100% 可并行的理想情况，以及 90%、75%、50% 可并行的情况。保留的串行部分限制了加速比。

读图：可并行比例为 90% 时，无限多处理器也只能接近 10 倍。观察蓝线逐渐变平，而不是盯着处理器数量增长。

手算一遍

串行程序耗时 100 s，其中 10 s 串行、90 s 可并行。使用 8 个处理器时，理想时间为 10 + 90/8 = 21.25 s，加速比约为 4.71；使用 32 个时为 12.8125 s，加速比约为 7.80。这里忽略通信、调度、同步和负载不均衡，实测可能更差。

强扩展 | Strong scaling

固定总问题大小，增加处理器，观察求解时间能否缩短。并行效率可定义为加速比除以处理器数；上例 8 个处理器的理想效率约为 58.8%。优化串行热点经常比继续加核心更有效。

1.2.2 Gustafson-Barsis 定律与弱扩展

核心段落 | 原书 p.13 · S006

Weak scaling represents the time to solution with respect to the number of processors for a fixed-sized problem per processor.

弱扩展描述的是：在每个处理器承担的问题规模固定时，求解时间如何随处理器数量变化。

E003 | SpeedUp(N) = N − S × (N − 1)

这是扩展问题规模后的加速比。讲解补充：此式中的 S 应按扩展后并行运行中的串行时间占比理解，不能把它与 Amdahl 式里固定问题的基准占比不加区分地混用。

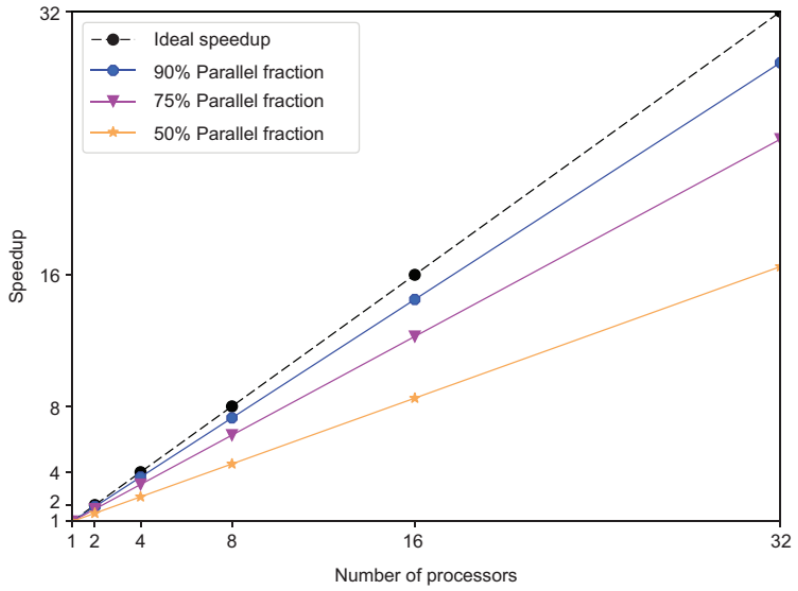


图 1.4 | 原书 p.13

Speedup for when the size of a problem grows with the number of available processors according to Gustafson-Barsis's Law is shown as a function of the number of processors. Lines show ideal speedup when 100% of an algorithm is parallelized, and for 90%, 75%, and 50%.

图注：当问题规模随可用处理器数量增长时，Gustafson-Barsis 定律给出的加速比。曲线分别对应理想的 100% 并行，以及 90%、75%、50% 并行的情况。

读图：这里增加的不只是处理器，也增加了工作量。它不推翻 Amdahl 定律，而是回答另一种工作负载下的问题。

同一个 N，不同的问题

若 N = 32、扩展后运行的串行占比为 0.1，则扩展加速比为 32 − 0.1 × 31 = 28.9。不要把它解释为“原来那项固定任务现在快了 28.9 倍”。它比较的是扩展后的工作量。

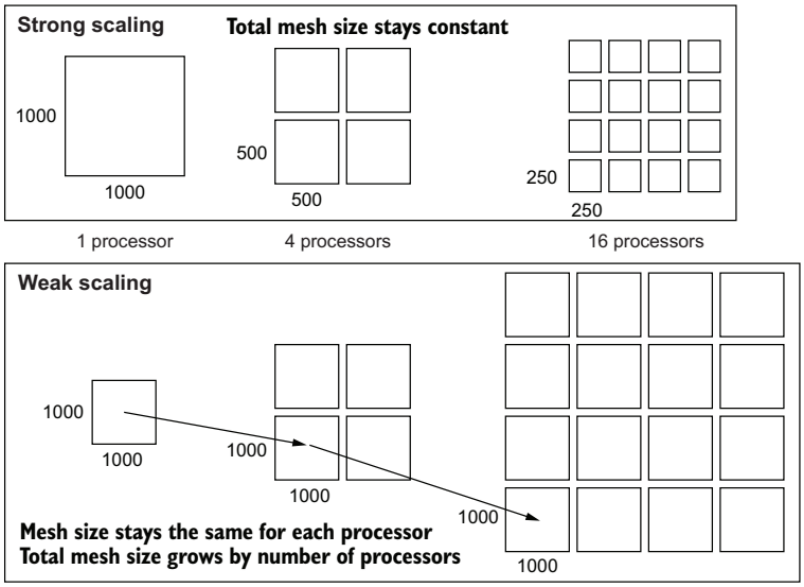


图 1.5 | 原书 p.14

Strong scaling keeps the same overall size of a problem and splits it across additional processors. In weak scaling, the size of the mesh stays the same for each processor and the total size increases.

图注：强扩展保持问题总规模不变，把它划分给更多处理器。弱扩展保持每个处理器上的网格规模不变，同时增大总规模。

读图：上半图总共仍是 1000 × 1000 个单元；16 个处理器各分 250 × 250。下半图每个处理器始终承担 1000 × 1000，因此总单元数随处理器数量增长。

内存扩展性：能运行是第一道门槛

运行时间扩展不好，任务只是慢；内存装不下，任务可能根本无法开始。原书因此强调分布数据。内存分布良好通常有利于性能，但并不保证通信和负载均衡也良好。

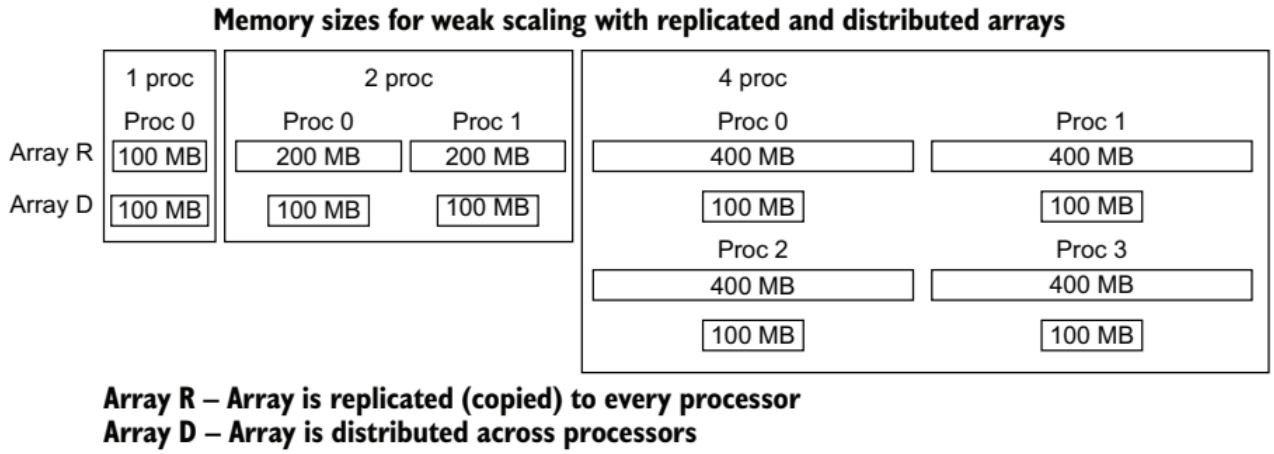


图 1.6 | 原书 p.14

Distributed arrays stay the same size as the problem and number of processors doubles (weak scaling). But replicated (copied) arrays need all the data on each processor, and memory grows rapidly with the number of processors. Even if the run time weakly scales (stays constant), the memory requirements limit scalability.

图注：在弱扩展中，问题规模与处理器数量同时翻倍时，每个处理器上的分布数组大小保持不变。但复制数组要求每个处理器都保存全部数据，内存需求会迅速增长。即使运行时间保持不变，内存需求仍会限制可扩展性。

读图：4 个处理器时，数组 D 每个进程仍是 100 MB；数组 R 每个进程达到 400 MB。复制部分的全系统占用是 4 × 400 MB。若全局数组规模随 N 线性增长且每个进程完整复制，总复制开销就随 N 的平方增长。

1.3 并行计算如何工作？ | 1.3.1 示例应用

核心段落 | 原书 p.15 · S007

Parallel computing requires combining an understanding of hardware, software, and parallelism to develop an application. It is more than just message passing or threading.

开发并行应用需要把对硬件、软件和并行性的理解结合起来。并行计算并不只有消息传递或多线程。

软件到硬件的路径

应用源代码表达工作和数据如何划分；编译器生成目标代码；操作系统与运行时将工作安排到硬件上。代码里写出了并行意图，不代表每一层都能自动获得理想性能。

第 1 步：离散化 | Discretize the problem

原书以喀拉喀托火山附近的二维区域为例。连续空间被切分为网格单元，每个单元存储波高、速度、烟尘浓度等数值。图像处理时也可把像素视为单元。离散化决定数据对象，后续才决定怎样分配计算。

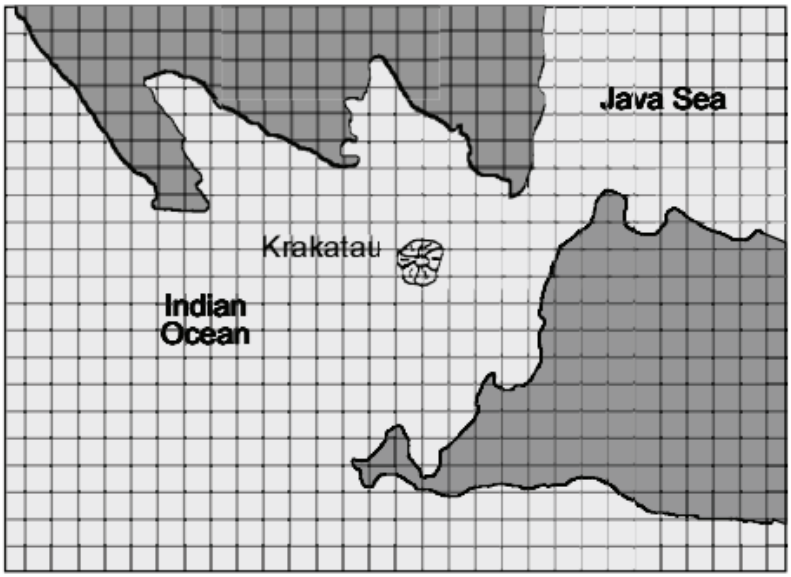


图 1.9 | 原书 p.17

The domain is discretized into cells. For each cell in the computational domain, properties such as wave height, fluid velocity, or smoke density are solved for according to physical laws. Ultimately, a stencil operation or a matrix-vector system represents this discrete scheme.

图注：区域被离散为单元。对计算域内的每个单元，根据物理定律求解波高、流速或烟尘密度等性质。最终可用模板运算或矩阵-向量系统表示这一离散方案。

读图：看网格划分，不要把“单元”误认为 CPU 核心。大量单元通常远多于硬件核心，因此一个核心会处理许多单元。

第 2 步：定义计算内核 | Define a computational kernel

内核是反复执行的核心计算片段。五点模板读取中心、左、右、上、下五个旧值，再计算中心的新值。不同中心的输出可以并行，但前提是它们读取同一份未被中途改写的旧数据。

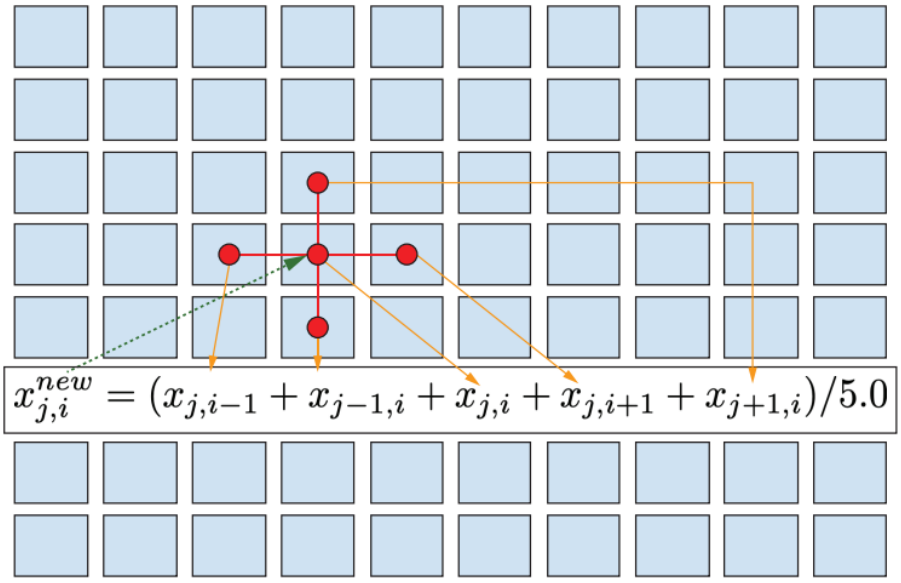


图 1.10 | 原书 p.18

A five-point stencil operator as a cross pattern on the computational mesh. The data marked by the stencil are read in the operation and stored in the center cell. This pattern is repeated for every cell. The blur operator, one of the simpler stencil operators, is a weighted sum of the five points marked with the large dots and updates a value at the central point of the stencil. This type of operation is done for smoothing operations or wave propagation numerical simulations.

图注：五点模板在计算网格上呈十字形。运算读取模板标出的数据，并把结果存入中心单元；对每个单元重复这一模式。模糊算子是一种较简单的模板运算，它对五个大圆点处的值进行加权求和，以更新中心值。这类运算用于平滑或波传播数值模拟。

读图：图内公式给五个点相同权重，除以 5.0。这里的“存入中心”应理解为新时间层的中心值；直接覆盖旧数组会改变相邻点随后读到的内容。

```
E004 | x_new[j,i] = (x[j,i-1] + x[j-1,i] + x[j,i] + x[j,i+1] + x[j+1,i]) / 5.0
```

图 1.10 内公式转写，非原书单独编号的公式。j 表示行，i 表示列；new 标记更新后的值。

原书解释的边界：PDE 中“偏”的含义

原书用 RGB 通道来帮助理解多变量变化，这只是类比。严格地说，偏导数是多自变量函数对其中一个自变量求导，其他自变量保持不变；单个标量场 u(x,y,t) 也可以满足偏微分方程，并不需要有多个颜色或多个物理场。五点平均是一个平滑示例，不能单独当作完整的海啸或流体求解器。

1.3.1 示例应用（续）：逐层加入并行

第 3 步：向量化 | Vectorization

一条向量指令对多个数据元素执行同一种操作。对 double 而言，256 bit 可以容纳四个元素。应把“4 个通道”与“四条任意指令同时执行”区别开；指令延迟也不一定只有一个时钟周期。

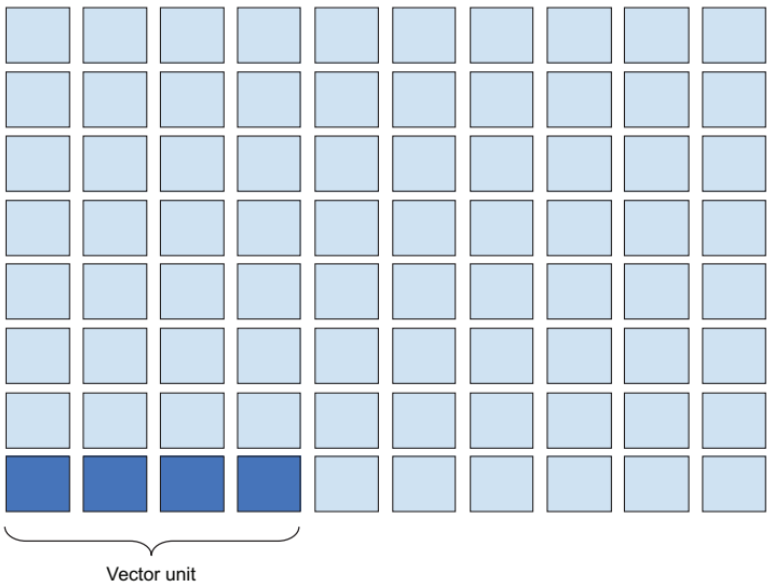


图 1.11 | 原书 p.19

A special vector operation is conducted on four doubles. This operation can be executed in a single clock cycle with little additional energy costs to the serial operation.

图注：一种特殊的向量操作作用于四个 double。原书指出，该操作可以在一个时钟周期内执行，相对串行操作只增加很少的能量开销。

读图：深色横排表示同时处理相邻数据。图注是概念化说明；真实指令的吞吐、延迟和功耗取决于硬件与指令类型，不能一概承诺“一周期完成”。

第 4 步：线程 | Threads

把不同的行交给不同线程，每个线程内部还可以进行向量化。因此“多线程”和“向量化”可叠加。共享旧数组没有问题，危险通常来自多个线程写同一位置，或一边读取一边改写别人需要的旧值。

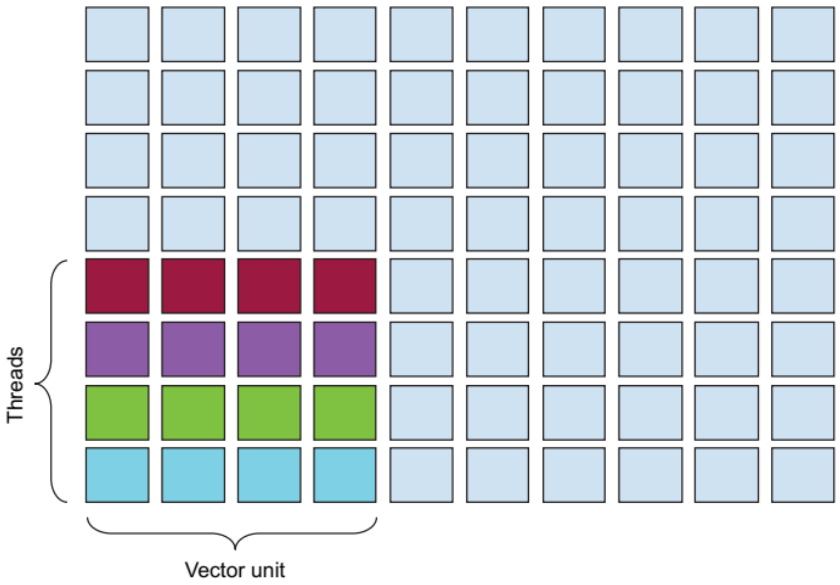


图 1.12 | 原书 p.20

Four threads process four rows of vector units simultaneously.

图注：四个线程同时处理四行向量单元。

读图：颜色区分线程，横向相邻单元体现向量通道。线程数量与物理核心数不是同一个概念。

第 5 步：进程 | Processes

再把网格分成不同子域交给独立进程。跨节点进程不能直接读取对方普通地址空间的数据。五点模板在子域边缘需要相邻子域的值，因此通常交换边界数据，形成 halo/ghost 区域；这部分通信在概念图里没有展开。

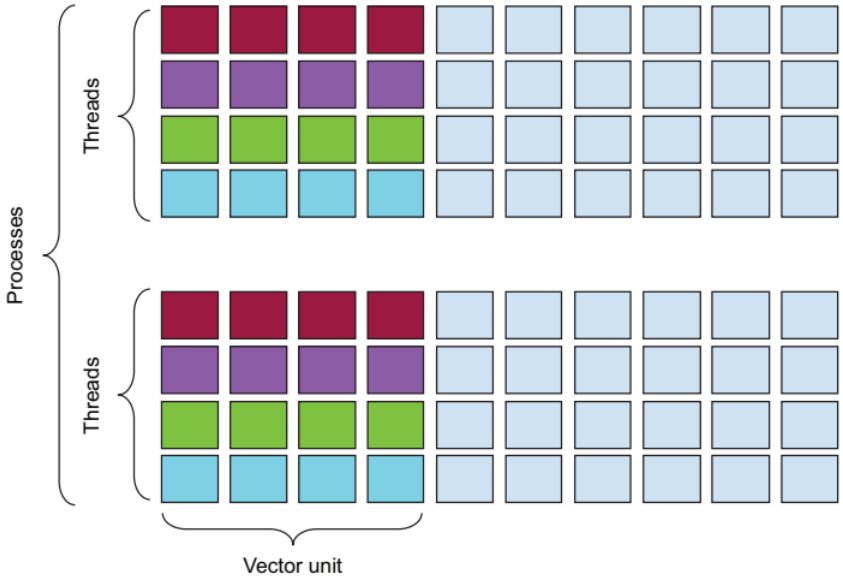


图 1.13 | 原书 p.20

This algorithm can be parallelized further by distributing the 4×4 blocks among distinct processes. Each process uses four threads, each handling a four-node-wide vector unit in a single clock cycle. Additional white space in the figure illustrates the process boundaries.

图注：把 4×4 数据块分配给不同进程，可以进一步并行化。每个进程使用四个线程，各线程处理四个位置宽的向量单元。图中增大的空白表示进程边界。

读图：这里的 four-node-wide 指示意图中四个数据位置，不是四个计算节点。原书 2 节点 × 4 核 × 4 通道 = 32 路，只表示理想并行机会。

第 6 步：卸载到 GPU | Off-loading

把二维数据切成 tile，交给 GPU 线程块处理。需要同时考虑计算量、设备内存、数据传输和线程组织。许多 GPU 工作线程用于覆盖等待延迟；线程数量不等于同一瞬间可执行的算术单元数量。



图 1.14 | 原书 p.21

On a GPU, the vector length is much larger than on a CPU. Here, 8×8 tiles are distributed across GPU work groups.

图注：GPU 上具有比 CPU 更大规模的数据并行能力。图中 8×8 数据块分配给不同 GPU 工作组。

读图：图中的 84 个 SM、32 个 FP64 单元和 8×8 tile 属于书中示例。tile 大小、warp 大小、SM 数量、算术单元数量是不同层次；不要直接把它们当作同一个“核心数”。本图第一句按含义翻译，GPU 的 SIMT 执行方式仍应与 CPU 向量寄存器宽度区分。

1.3.2 异构并行系统的硬件模型

核心段落 | 原书 p.22 · S008

Compute nodes can be connected to each other with one or more networks, sometimes called an interconnect. Conceptually, a node runs a single instance of the OS that manages and controls all of the hardware resources.

计算节点可以通过一个或多个网络互连。概念上，一个节点运行一个操作系统实例，由它管理和控制节点的硬件资源。

分布式内存架构 | Distributed memory architecture

每个节点具有自己的本地内存，通过网络与其他节点通信。优势是可以增加节点来扩展总资源；代价是程序需要显式划分数据，并为跨节点访问安排通信。

共享内存架构 | Shared memory architecture

同一节点中的多个核心可以访问进程共享的数据。编程上更直接，但同时更新同一变量可能出现数据竞争；共享带宽和同步开销也会限制扩展。增加核心数并不会自动增加物理内存容量。

向量单元 | Vector units

把多个数据元素放入向量通道，执行同一种运算。这提高单位指令处理的数据量。数据连续、循环依赖少、分支少时通常更容易表达向量化，但具体收益需要编译器分析和测量。

加速设备 | Accelerator device

集成 GPU 与独立 GPU 是不同部署方式。独立 GPU 通常有自己的设备内存，经总线或专用互连与主机交换数据。GPU 的 SM 是包含调度器、执行单元、寄存器和共享内存等资源的组合，不能等同于一个 CUDA core。[R2]

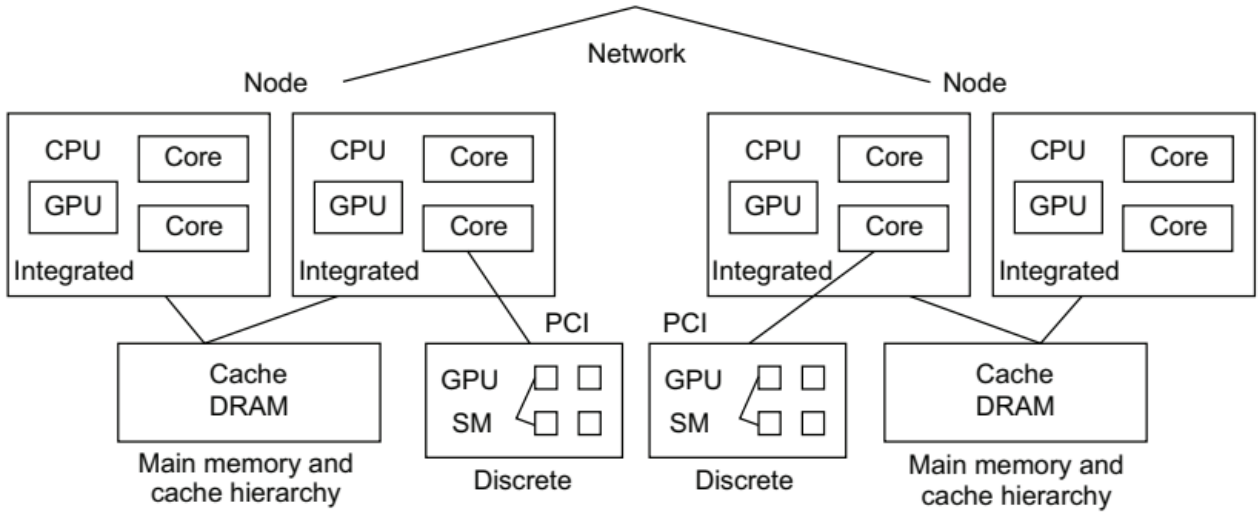


图 1.19 | 原书 p.25

A general heterogeneous parallel architecture model consisting of two nodes connected by a network. Each node has a multi-core CPU with an integrated and discrete GPU and some memory (DRAM). Modern compute hardware normally has some arrangement of these components.

图注：通用异构并行架构模型：两个节点经网络相连。每个节点包含多核 CPU、集成与独立 GPU，以及 DRAM 等内存资源。计算硬件可以由这些组件以不同方式组合而成。

读图：先辨认 Network、Node、CPU、Core、Cache/DRAM、GPU，再沿连线观察数据移动。图是教学模型，不意味着所有计算机都有这些部件，或具有完全相同的连接结构。

通用异构架构 | General heterogeneous architecture

异构表示存在不同类型的计算资源。NUMA 表示共享地址空间中，不同内存位置的访问成本可能不同；“能访问”不等于“访问一样快”。原书本章只给出简化模型，缓存层次留到第 3、4 章深入。

1.3.3 异构并行系统的软件模型

核心段落 | 原书 p.26 · S009

Processes are an OS concept, and processors are a hardware component.

进程是操作系统中的概念，而处理器是硬件组件。

进程并行：消息传递 | Process-based parallelization

进程拥有独立的地址空间，MPI 提供消息传递接口。rank 是进程在某个通信器内的编号，并不是物理核心编号。进程数可以超过核心数；操作系统会调度它们，但这不等于硬件执行资源随之增加。

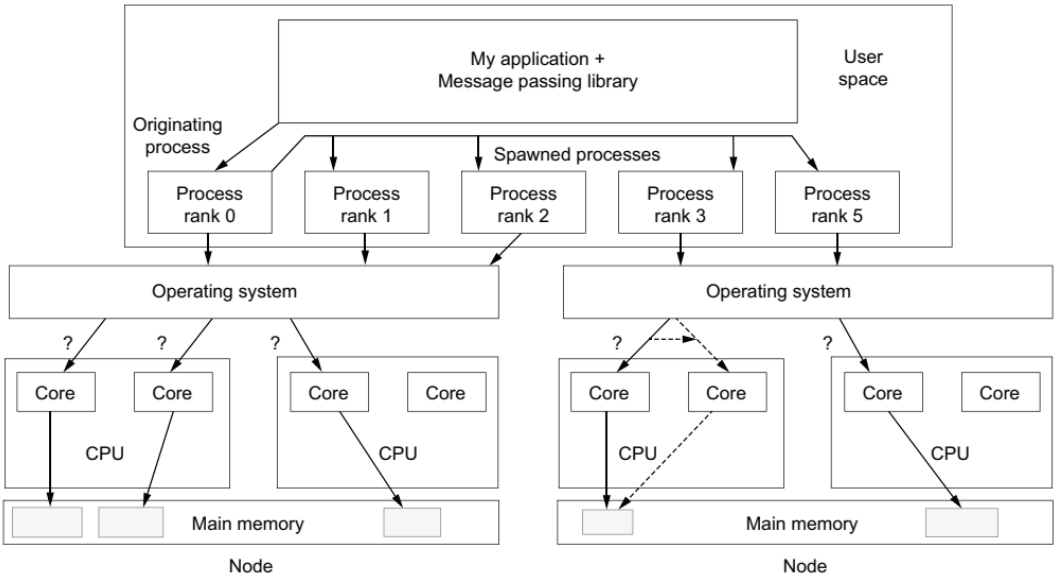


图 1.20 | 原书 p.26

The message passing library spawns processes. The OS places the processes on the cores of two nodes. The question marks indicate that the OS controls the placement of the processes and can move these during run time as indicated by the dashed arrows. The OS also allocates memory for each process from the node's main memory.

图注：消息传递库创建进程，操作系统将进程安排到两个节点的核心上。问号表示进程放置由操作系统控制，虚线表示运行中可能迁移。操作系统也从节点主存为进程分配内存。

读图：这是书中的抽象示意。实际 MPI 作业常由启动器和调度系统创建进程，MPI_Init 不意味着总会在程序内部创建全部进程。绑定和亲和性会影响实际放置。

线程并行：共享内存 | Thread-based parallelization

一个进程内有多个线程，每个线程有自己的执行状态，能够访问共享数据。共享带来便利，也意味着需要约束写操作。OpenMP 可把循环迭代分配给线程。线程本身不会让一个普通进程的地址空间自动跨越多台机器。

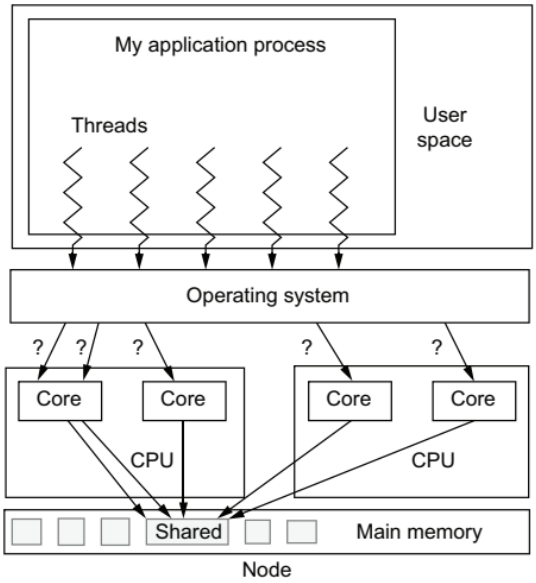


图 1.21 | 原书 p.27

The application process in a thread-based approach to parallelization spawns threads. The threads are restricted to the node's domain. The question marks show that the OS decides where to place the threads. Some memory is shared between threads.

图注：在线程并行方式中，应用进程创建线程，线程限制在节点范围内。问号表示操作系统决定线程放置位置。线程之间共享部分内存。

读图：与图 1.20 对比：线程在一个进程内共享数据，多个进程通常需要通信来交换数据。

向量化：一条指令处理多个数据 | Vectorization

可通过编译器自动分析、OpenMP SIMD 指令或更低层的内在函数表达。编译选项影响生成的目标指令；源码完全相同，也可能因编译器和目标硬件不同而性能不同。

流处理：使用专门处理器 | Stream processing

原书以 GPU 为主：把数据与计算内核卸载到设备，计算后在需要时传回结果。若每次只计算一点点却频繁往返传输，通信可能抵消计算收益。

术语边界：分布式与并行

原书把分布式计算放在其并行计算框架内介绍。更广义的语境下，两者侧重点不同：分布式关注分离组件的协作，并行关注同时执行以改善性能。分布式系统可能主要为了可用性与容错，而不只是加速。

1.4 并行方式的分类

核心段落 | 原书 p.29 · S010

Be aware that though the taxonomy is useful, some architectures and algorithms do not fit neatly within a category.

需要注意，尽管这种分类很有用，有些架构和算法并不能整齐地归入某一类别。

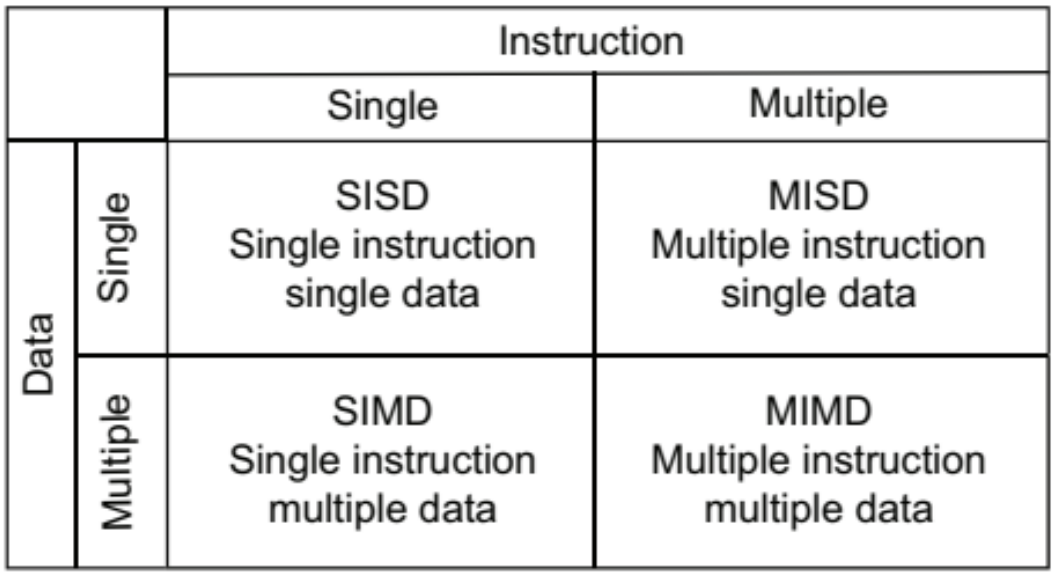


图 1.24 | 原书 p.29

Flynn's Taxonomy categorizes different parallel architectures. A serial architecture is single data, single instruction (SISD). Two categories only have partial parallelization in that either the instructions or data are parallel, but the other is serial.

图注：Flynn 分类用于区分不同的并行架构。串行架构属于单指令流、单数据流（SISD）。其中两类只在指令流或数据流中的一个维度上具有多路结构。

读图：横轴是指令流数量，纵轴是数据流数量。先弄清分类坐标，再记缩写。

四种类别

SISD：单指令流、单数据流，可用于理解传统串行执行。SIMD：单指令流、多数据流，典型例子是向量加法。MISD：多指令流、单数据流，实际分类较少见。MIMD：多指令流、多数据流，多个核心可独立执行不同指令序列。

SIMT 与 SIMD 的关系

GPU 常以 SIMT（单指令、多线程）描述编程与执行方式：程序写成每个线程处理一项工作，硬件以 warp 等组来组织执行。它与 SIMD 有联系，但不能把 SIMT、线程块和 warp 混成一个概念。同组线程进入不同条件分支时，可能降低执行效率。

原书类比的边界

多个处理器对同一输入运行同一份程序进行冗余校验，不一定就严格构成“多指令流”。使用 MISD 标签时，应看实际指令流组织，不能只看有没有重复计算。

1.5 并行策略

核心段落 | 原书 p.30 · S011

Essentially, each process executes the same program but operates on a unique subset of data as illustrated in the upper right of figure 1.25.

本质上，每个进程执行同一个程序，但处理不同的数据子集，如图 1.25 右上方所示。

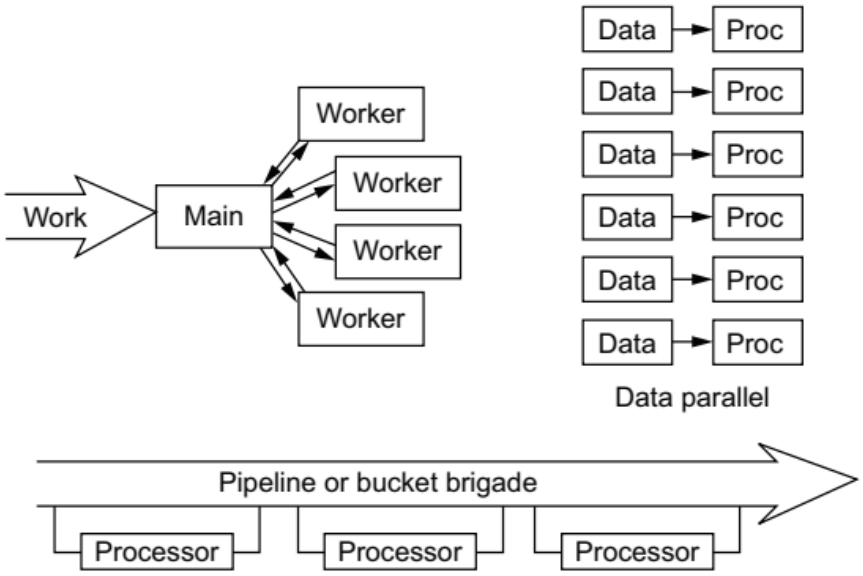


图 1.25 | 原书 p.30

Various task and data parallel strategies, including main-worker, pipeline or bucket-brigade and data parallelism

图注：不同的任务并行与数据并行策略，包括主控-工作者、流水线或接力式处理，以及数据并行。

读图：右上：相同操作作用于不同数据；左侧：主控分配任务给工作者；下方：不同阶段依次加工数据，但多个数据项可处在不同阶段。

数据并行 | Data parallelism

把同一计算应用于不同数据块。网格更新、数组运算和批量图像处理都可采用这种方式。若每块工作量接近，静态划分通常很直接；若工作量差别很大，就需要更合适的调度。

任务并行 | Task parallelism

不同任务可以做不同的事。主控-工作者适合动态领取任务，但主控本身可能成为瓶颈。流水线把处理分为多个阶段，稳定吞吐量受最慢阶段制约。单个任务的延迟与整批任务的吞吐量需要分别讨论。

策略可以组合

一个 MPI 进程负责一片数据，进程内用 OpenMP 多线程，每个线程再向量化；也可以在多个 GPU 间分配独立任务。组合越多，数据归属、同步与资源分配越需要明确。

1.6 并行加速比与对比加速比：两种不同度量

核心段落 | 原书 p.31 · S012

For example, comparing a GPU run time to a serial CPU run is not a fair comparison between a multi-core CPU and the GPU.

例如，用 GPU 的运行时间与串行 CPU 的运行时间比较，并不能公平地比较多核 CPU 与 GPU 的能力。

并行加速比 | Parallel speedup

从单一串行基线到并行版本，回答“引入这种并行方式后快了多少”。如果串行 CPU 是 100 s、GPU 是 2 s，那么这一基线下是 50 倍。

对比加速比 | Comparative speedup

比较两种合理利用资源的实现，回答“在给定资源条件下哪个方案更快”。若多核 CPU 为 5 s、GPU 为 2 s，则 GPU 相对多核 CPU 是 2.5 倍。它与前面的 50 倍并不矛盾，因为分母不同。

怎样读性能数字

至少同时记录输入规模、数值精度、正确性标准、CPU/GPU 型号与数量、线程/进程数、编译器和选项、计时范围，以及是否包含传输和初始化。原书还建议注明硬件发布年份和档次，减少以较弱旧 CPU 对比强大新 GPU 的误导。

应用时间与内核时间

只计 GPU 内核的 2 ms，不代表整项任务也只需 2 ms。若数据搬运 20 ms、初始化 10 ms，端到端收益需要把这些时间纳入。应先明确问题是稳态迭代速度，还是一次完整作业的用时。

1.7 本书将教会你什么？

核心段落 | 原书 p.32 · S013

To fully benefit from this book, readers should be proficient programmers, preferably with a compiled, HPC language such as C, C++, or Fortran.

为了充分受益于本书，读者应具备熟练编程能力，最好熟悉 C、C++ 或 Fortran 等用于高性能计算的编译型语言。

本章学习目标

能够区分线程、进程、物理核心与向量通道；能用 Amdahl 定律估计固定问题的收益；能解释弱扩展为何允许解决更大问题；能看出五点模板的数据依赖；能辨认性能比较使用的基线。

后续章节路线

第 2-5 章：并行化规划、性能分析、数据布局、并行算法。第 6-8 章：向量化、OpenMP、MPI。第 9-13 章：GPU 架构、编程模型、指令式与底层编程、性能工具。第 14-17 章：亲和性、作业调度、并行文件操作、开发工具。

1.7.1 延伸阅读 | Additional reading

原书列出 Blaise Barney 的 Introduction to Parallel Computing，来自 Lawrence Livermore National Laboratory。这里保留原书阅读线索；本讲义不把该外部教程当作已逐页核验的来源。代码说明的补充依据见文末 R1-R4。

教学补充 A | 把“并行性”写成 C / OpenMP

示例用途

第 1 章没有完整程序清单。以下代码是为讲解补写的示例，不是原书代码翻译。它对 8 个元素执行数组加法，再归约求和；如此小的数组只适合理解语法，不适合测加速比。完整文件为 vector_add.c。

```c
#include <stdio.h>

int main(void) {
    enum { N = 8 };
    double a[N], b[N], c[N];
    for (int i = 0; i < N; ++i) {
        a[i] = i;
        b[i] = 2.0 * i;
    }
    #pragma omp parallel for simd
    for (int i = 0; i < N; ++i) {
        c[i] = a[i] + b[i];
    }
    double sum = 0.0;
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < N; ++i) {
        sum += c[i];
    }
    printf("sum = %.1f\n", sum);
    return 0;
}
```

语法逐行解释：C 的基础部分

#include <stdio.h> 引入标准输入输出声明，以便调用 printf。int main(void) 表示主函数无参数并返回整数。enum { N = 8 } 提供一个编译期整数常量。double a[N] 声明 N 个双精度数；a[0] 是第一个元素，最后一个是 a[N-1]。花括号包围语句块，分号结束普通语句。

语法逐行解释：循环与下标

for (int i = 0; i < N; ++i) 的三部分分别是初始化、继续条件和每轮后的更新。++i 把 i 增加 1；i < N 防止访问 a[N]。赋值 a[i] = i 会把整数转换为 double；2.0 * i 是浮点乘法。c[i] = a[i] + b[i] 每轮写不同元素，因此各次迭代可独立执行。

语法逐项解释：OpenMP 指令

#pragma 是给编译器的指令入口；omp 指定 OpenMP；parallel 建立线程团队；for 把循环迭代分配给线程；simd 表达循环迭代可进行 SIMD 执行。[R4] 编译器与运行时决定具体映射，不能由这一行推断一定使用多少线程或多少向量通道。

为什么求和需要 reduction

sum += c[i] 等价于读取 sum、加上 c[i]、再写回。多线程直接修改同一个 sum 会发生竞争。reduction(+:sum) 为参与者提供私有累加结果，并按加法规约合并。[R3] 循环变量 i 受 OpenMP 循环规则管理，不是所有线程共同推进同一个计数器。

输出、构建和验证范围

printf 中 %.1f 表示保留一位小数，\n 是换行转义，return 0 表示正常退出。若使用支持 OpenMP 的 GCC，可执行 gcc -O2 -fopenmp vector_add.c -o vector_add；Windows PowerShell 下运行 .\vector_add.exe。预期输出 sum = 84.0。本次未找到可直接调用的 C/OpenMP 编译器，因此未宣称已编译运行；已复核算式与数据依赖。

教学补充 B | 五点模板：为什么要两份数组？

```
#define NX 6
#define NY 6

void step(const double old[NY][NX],
          double next[NY][NX]) {
    #pragma omp parallel for
    for (int j = 1; j < NY - 1; ++j) {
        #pragma omp simd
        for (int i = 1; i < NX - 1; ++i) {
            next[j][i] = (old[j][i-1]
                + old[j-1][i] + old[j][i]
                + old[j][i+1] + old[j+1][i]) / 5.0;
        }
    }
}
```

#define、const 和二维数组

#define NX 6 是预处理宏替换，没有结尾分号。void 表示函数不返回值。const double old[NY][NX] 表示函数通过这个参数读取二维数组，不能通过它修改元素；它不会复制一整份数组。作为函数参数，二维数组形参调整为指向一行的指针，编译器需要 NX 才能确定行跨度。next 是可写输出数组。

边界与数据布局

j 从 1 到 NY-2，i 从 1 到 NX-2，保证上下左右下标不越界。四周边界由调用者预先设置，step 只更新内部点。C 二维数组采用行优先布局，i 连续变化访问同一行中的相邻元素，因此将 i 放在内层适合连续访问。

并行为什么安全

假设 old 和 next 是两个互不重叠的数组，且计算期间 old 不被其他代码修改。所有线程只读 old，每个内部 next[j][i] 由唯一一次迭代写入。外层按行分给线程，内层表达 SIMD。并行循环结束的隐式同步保证调用者随后可以使用完整的新结果。

如果写成 old[j][i] = ... 会怎样

相邻迭代会读到已更新和未更新的混合值。串行结果可能因此依赖遍历顺序；并行版本还可能产生读写竞争。双缓冲让“旧时间层”和“新时间层”分离，避免把算法无意中改成另一种更新方式。注意 const 并不能阻止通过别的别名修改同一内存，所以两个参数不重叠是这里的前提。

一个可核对的输入

完整示例 stencil.c 把 6×6 旧数组设为 0，只令 old[2][2] = 10。计算一步后，next[2][2]、next[2][1]、next[2][3]、next[1][2]、next[3][2] 均为 2.0，其他内部值为 0，边界保持 0。中心值“扩散”到五个位置，正对应图 1.10。此例只验证一次平滑更新，不代表完整物理模型。

1.7.2 练习｜题意与解题思路

练习 1：日常并行

题意：再找一些日常并行操作，分类并说明优化目标，尝试计算加速比。参考：四个窗口处理 100 份互不依赖的业务。若每份固定 1 分钟且均匀分配，串行 100 分钟，并行 25 分钟，理想加速 4 倍；若有公共审核环节，则应另计串行与等待开销。

练习 2：你的设备有哪些并行资源？

题意：识别桌面、笔记本或手机上的并行硬件，比较理论并行与串行能力。先记录物理核、逻辑处理器、向量指令支持、GPU 和内存，再说明哪个程序能使用这些资源。不要把不同类型“核心数”直接加总，也不要把超线程机械地算成两倍吞吐。本讲义未读取你的机器硬件配置。

练习 3：给超市示例分类

题意：指出图 1.1 中的并行策略及图中未展现的策略。多个窗口处理不同顾客可视作任务分配；相同结账流程作用于不同购物篮也有数据并行视角。若把扫码、装袋、付款分成多个阶段，并允许不同顾客占据不同阶段，就形成流水线。分类取决于建模粒度。

练习 4：每天处理 1000 张图像

题意：每张 4 MiB，串行需 10 分钟；每节点 16 核、16 GiB 内存，如何设计？需求增至 10 倍怎么办？参考：若图像互不依赖，优先按图像分任务，用工作队列分配到核心。一个节点理想完成时间为 ceil(1000/16) × 10 = 630 分钟。输入数据共 4000 MiB，而计算中间数据的额外内存仍需单独估计。

练习 4 的扩展计算

需求增至 10000 张时，单节点需 6250 分钟。若要求 24 小时内完成，单核每天最多处理 144 张，理论至少 ceil(10000/144)=70 个核心，即至少 5 个 16 核节点。5 节点的理想完成时间为 ceil(10000/80) × 10=1250 分钟。均假设核心独占任务、任务耗时一致，且忽略 I/O、调度和资源竞争；实际部署要测量并留余量。1 MiB = 2^20 bytes，1 GiB = 2^30 bytes。

练习 5：GPU 至少要快多少才更节能？

题设 CPU 为 130 W，GPU 为 300 W。按题目简化模型，GPU 更节能要求 300 × T_GPU < 130 × T_CPU，因此 T_CPU/T_GPU > 300/130 ≈ 2.31。恰好 2.31 倍只是能耗相等，严格更节能要超过它。原书第 10 页同型号例题写 120 W，而第 33 页此题写 130 W；这里按各处题设分别计算，未擅自统一。整机比较还应计入 GPU 主机等功耗。

Summary｜本章要点

可并行的工作量、串行瓶颈、数据归属与移动成本共同决定收益。先得到正确结果，再验证性能；报告加速比时说清基线。能用两个小程序解释“每轮读什么、写什么、是否互相依赖”，就已掌握后续 OpenMP、MPI 和 GPU 学习最重要的起点。

补充来源与交付说明

R1 · Intel：超线程共享物理核心资源

https://www.intel.com/content/www/us/en/support/articles/000098959/processors/intel-xeon-processors.html

R2 · NVIDIA：CUDA C++ Programming Guide，SM 与线程执行组织

https://docs.nvidia.com/cuda/archive/12.9.0/pdf/CUDA_C_Programming_Guide.pdf

R3 · OpenMP 5.2：reduction 子句

https://www.openmp.org/spec-html/5.2/openmpsu52.html

R4 · OpenMP 5.2：规范目录，parallel、worksharing-loop、simd 与组合指令

https://www.openmp.org/spec-html/5.2/openmp.html

译文与原图的关系

17 幅图均直接裁自用户提供的原书，未重新绘制。原图中的英文与历史参数保留；中文图注和讲解紧随图片。正文摘译只覆盖选定核心段落，其余内容按小节解释，具体范围记录在 translation_notes.md。

代码验证状态

两个示例已进行数据依赖和预期数值复核。本次环境未找到可直接调用的 C/OpenMP 编译器，未完成实际编译和多线程运行，不把说明中的预期输出写成实测结果。

文件组成

paper.md：可检索、可修改的双语讲义。source_map.json：段落、图注、公式与页码映射。translation_notes.md：翻译范围、原书简化与数值差异记录。assets：17 幅原图。vector_add.c、stencil.c：独立示例。